

chama.

Advanced technical overview

A protocol for people trading and saving together.

September 2026 | 6.4 release line | Prepared for 6.4.3

Chama combines Nostr coordination, Fedimint ecash, Lightning payments, and optional on-chain Bitcoin escrow. The application has no central account server. Its clients validate trade history and coordinate actions between named participants.

This edition describes the current implementation and separates active product paths from disabled writers and future design work. It replaces the v5.1 overview, which centered commitment bonds and overstated the guarantees of short-lived ecash.

The essential boundary: identity, escrow state, wallet state, and a confirmed external payout are different things. Recovering one does not automatically recover the others.



BUYER



SELLER



ARBITER

People agree. Clients verify.

Coordination

Nostr keys identify participants. Signed events carry offers and trade history over relays. The parser and reducer validate authorship, roles, payloads, and legal transitions before a client accepts a new state. A relay response is evidence to examine, not authority to settle a trade.

Public offers and event metadata remain observable. Sensitive escrow messages use production encryption and recipient envelopes; trade chat is encrypted. Encryption does not conceal every relationship or timestamp, and relay availability can leave a client with incomplete history.

Money interfaces

Fedimint issues and redeems federation-specific bearer ecash. Lightning connects external wallets and federation gateways. Native on-chain escrow uses Bitcoin outputs with committed spending conditions. A local-currency transfer or physical delivery is performed by the trading parties or an external provider, not by the Nostr event engine.

Client surfaces

The React/Vite client runs as a web/PWA app, in an Android Capacitor shell, and in a Tauri desktop shell. Browser wallets use the WASM adapter. Packaged native integrations use the Rust Fedimint bridge. StartOS packages this application separately and operates its bridge with persistent server storage.

Trust boundaries remain: federations and their guardians, relays, local device security, counterparties, and external payout services. "No Chama custodian" does not mean a system without dependencies or risk.

Source: [src/escrow-engine/event-parser.ts](#) · [src/escrow-engine/state-machine.ts](#) · [src/escrow-engine/encryption-config.ts](#) · [src/fedimint/native/fedimint-bridge/](#)

Agreement is not yet a payout.

The normal path

A listing establishes the terms and permitted roles. Participants join, the funding side locks the ecash, and buyer and seller perform the agreed exchange. Valid votes resolve a release or refund. The entitled recipient then claims and routes the payout. A settled outcome, redeemed ecash, and a paid external invoice are distinct milestones.

Ecash protection uses Shamir secret sharing with a threshold of two among buyer, seller, and the seated arbiter. Holder-specific encrypted shares restrict what a single recipient learns. Normal settlement combines the two principals' agreement; an arbiter can participate in a dispute or protocol-defined expiry recovery. Current authority comes from committed trade state, not a person's past presence in a community pool.

What the threshold does not prove

Fedimint notes are bearer instruments. Splitting knowledge of a note does not make it a Bitcoin script: the original funder can have known the unsplit note, and two share holders can combine knowledge. Application validation constrains lawful outcomes, but cannot make all malicious clients obey. Federation trust, note custody, and participant collusion remain material assumptions.

Money operations are asynchronous

Funding and locking are coordinated with durable recovery state. Network effects do not become one atomic transaction across a wallet, federation, and Nostr relay. A request may succeed even if its response is lost. Pending operation identifiers and reconciliation prevent an unknown result from being treated as permission to pay again.

Operational rule: preserve the original operation and wallet state. An error message alone is not evidence that no money moved.

Source: [src/escrow-engine/escrow-client.ts](#) · [src/payments/fund-and-lock.ts](#) · [src/payments/payout-journal.ts](#) · [src/bond-multisig/onchain-escrow.ts](#)

A separate enforcement boundary.

Native Bitcoin escrow is available as an opt-in trade substrate. It is distinct from withdrawing Fedimint funds to a Bitcoin address: the former locks the trade in a UTXO; the latter is only a payout destination after an ecash claim.

Three spending paths

Cooperative: buyer and seller sign the agreed transaction together.

Dispute: the committed script provides a two-of-three participant branch. Where a relative block delay is committed, Bitcoin enforces that delay; clients must inspect the actual terms rather than infer them from a UI label.

Timeout refund: after the committed absolute block height, the funder can use the refund branch. This is an explicit recovery condition, not an unconditional promise that every escrow always requires two signatures.

The Taproot construction uses an unspendable internal key and script paths. The client derives and checks the expected scripts, keys, address, network, and funding outpoint. Settlement transport carries transaction material for verification and signing; relay publication is not a Bitcoin confirmation.

Different costs and failure modes

The current escrow network constant is mainnet. Bitcoin confirmation timing, transaction fees, funding detection, and delayed refund conditions matter. There is no fixed wall-clock guarantee for a block-height timeout. A funded transaction and an unconfirmed replacement must be reconciled against chain state.

Do not confuse: an on-chain trade escrow, an owner's commitment bond, and a federation's on-chain deposit or withdrawal are three different constructions with different beneficiaries and recovery rules.

Source: [src/bond-multisig/onchain-escrow.ts](#) · [src/bond-multisig/onchain-escrow-funding.ts](#) · [src/bond-multisig/onchain-escrow-settle.ts](#) · [src/escrow-engine/onchain-settlement-transport.ts](#)

What runs today. What comes next.

Active: fill-or-refund

The circle parent records equal share size, minimum membership, an optional cap, the fill deadline, and the round end. Members fund separate child escrows. If the threshold is reached in time, the round runs; otherwise it fails to fill. Current share-v1 rules permit refund outcomes, returning each member's own principal rather than transferring the pot to another member. External funding and payout fees remain separate.

The host organizes the round but cannot take a funded seat while the ring writer is disabled. The surface exposes claim readiness and claimed states; a host can prepare another round or retry a failed fill. A new round is a new commitment, not an automatic reinvestment.

Implemented readers, disabled writers

Ring witnessing and rotation-v2 logic are present for validation and replay, but `CHAMA_RING_WRITER_ENABLED` and `CHAMA_ROTATION_ENABLED` are false. The rotating-pot product is therefore not active in this release. New round numbers in the existing flow do not turn it on.

The future rotation design seals membership, derives collection order, and carries settlement evidence so replay evaluates the commitment rather than a client's accidental relay view. Host participation, rotating collection, and standing penalties belong to that gated design. Its specifications are not evidence of a live product switch.

Simulation

Sim mode uses a mock wallet and tags its events. Incoming-event policy separates simulation from real trades. A ten-minute circle test drive allows five minutes to fill and five to run. Short simulated rounds bypass the real pot-circle duration floor; that exception is not available to untagged real-money events.

Source: [src/chama/](#) · [src/escrow-engine/experimental-escrow-features.ts](#) · [src/chama/policy.ts](#) · [src/sim/simMode.ts](#) · [docs/chama-rotation-v2-spec.md](#)

Your identity is not an ecash backup.

Keep four records distinct

Identity key: the Nostr secret controls signing and access to encrypted coordination. A public npub is not a recovery secret. Restoring the identity does not reconstruct every local wallet database or pending payment operation.

Trade history: relay events and local caches support replay. Missing events can make a view incomplete. A status reconstructed from history is not proof that a separate wallet transfer finished.

Wallet state: browser Fedimint state is device-local. Native bridge state belongs to its own storage location; a StartOS bridge persists it on the server volume. Using the same Nostr identity elsewhere does not turn these into a single shared wallet.

Bearer notes and operation journals: exported ecash is spendable by whoever possesses it. Pending spends, recovery copies, and payout records support reconciliation. Protect them independently of identity backup.

Fail closed

Do not replace, rotate, or delete a possibly funded wallet to make startup work. Do not clear browser data, reinstall, or discard a native volume while money or unresolved operations remain. Federation routing must preserve the relationship between notes, their issuing federation, and the wallet that can recover them.

A claim can require attention because redemption or the outbound payment is still uncertain. Inspect the destination and existing operation before retrying. Automatic expiry recovery depends on eligible clients, sufficient history, and connectivity; an always-on headless healer is roadmap work, not a guaranteed service in this release.

Product intent is not a zero-balance invariant: claim or export promptly, but account for local funds, fee reserves, and unfinished operations between trades.

Source: [src/fedimint/browser-wallet-recovery-journal.ts](#) · [src/fedimint/native-bridge-adapter.ts](#) · [src/payments/payout-journal.ts](#) · [src/escrow-engine/escrow-client.ts](#) · [docs/v6.4-runway.md](#)

Commitment is not compensation.

Owner-held Bitcoin bonds

A commitment bond locks the owner's Bitcoin until an absolute block height using a Taproot timelock leaf. The owner alone can reclaim after maturity; it is not a standing committee's ecash pool. Clients recompute the bond construction and check chain funding and maturity rather than trusting a Nostr announcement.

A verified bond can qualify an identity for product privileges. It is not proof of honesty, a slashable insurance fund, or permission to move another participant's money. Arbiter obligations and payment eligibility must come from the specific trade's accepted state.

Premiums in this implementation

The current optional settlement premium is 0.25% per principal for an eligible bonded arbiter seated in the trade. Eligibility uses the trade's committed bonded subset, and notes below the minimum size are skipped. Premium delivery uses a separate encrypted ecash event. Do not present this as a reimbursement policy or imply that every community member earns it.

The public, vouched arbiter roster and dispute-only premium doctrine in newer design notes are future work. They do not replace the current payment behavior merely by appearing in a specification.

Stores and orders

A storefront is an offer; individual purchases can be child escrows with their own participants and money. Stock and active orders derive from child state. Listing continuity and renewal must not resurrect a funded or cancelled trade or silently extend a locked trade's timeout. The public presence of a store is not evidence that every order settled.

Keep the boundaries visible: identity and reputation describe a person; a bond describes a commitment; a storefront advertises terms; a trade's committed state determines its obligations.

Source: [src/bond-multisig/commitment-bond.ts](#) · [src/arbiters/arbiter-premium.ts](#) · [src/arbiters/bonded-stamp.ts](#) · [src/escrow-engine/storefront.ts](#) · [docs/arbiter-roster-spec.md](#)

Read the implementation, not the teaser.

Current product gates

Circles and the guided live-trade surface are enabled. New trade slicing and the guided slice chooser are disabled. Stack and the new XBT market should not be described as active just because a teaser or implementation module exists. Disabled creation paths do not justify deleting readers or recovery paths for historical, possibly funded trades.

Event vocabulary

Core escrow events include CREATE (38100), JOIN (38101), LOCK (38102), VOTE (38103), RESOLVE (38104), CLAIM (38105), COMPLETE (38106), CANCEL (38107), and CHAT (38108). READY and KICK allocations are retired. Extensions include SUBSCRIBE (38111), PERIOD_RELEASE (38112), PREMIUM (38113), SETTLEMENT (38114), PLAN_START (38115), and CHILD_KEY (38116).

The allocation of an event kind does not mean its writer is currently enabled. Payload version, feature gates, committed roles, encryption rules, and replay validation together determine what is acceptable.

Distribution and verification

The authoritative application repository is [jesuspirate/chama](#). Signed application releases use vX.Y.Z tags. The separate [Start9-Community/chama-startos](#) package pins the application through its submodule and owns packaging versions and builds. The retired [Start9-Community/chama](#) fork is not a release source.

The release entry point is `npm run ship`. Typecheck, tests, build, and repository hygiene establish release checks; they are not proof against every operational failure or malicious participant. Web, landing, Android, desktop, and listing publication are separate distribution concerns.

Reading map

Start with [README.md](#) and [PHILOSOPHY.md](#) for intent; use [experimental-escrow-features.ts](#) for gates, [types.ts](#) for event definitions, and [parser/reducer](#) tests for accepted behavior. Design documents describe direction where writers remain off. Consult [docs/RELEASING.md](#) for publishing.

Source: [src/escrow-engine/types.ts](#) · [src/escrow-engine/experimental-escrow-features.ts](#) · [README.md](#) · [PHILOSOPHY.md](#) · [docs/RELEASING.md](#)